

Yes. Here is a revised plan built around online resources you can actually follow.

Goal

Use this plan to understand the Rust concepts, architecture, and use cases in this codebase:

- `G:/forge/lib` : domain models, services, repository traits, events.
- `G:/forge/arma/crate` : Arma extension boundary, command routing, persistence, async runtime.

Phase 1: Rust Foundations

Use these first:

- [The Rust Programming Language](#)
Main book. It now targets Rust 2024 edition, which matches this repo's `edition = "2024"` setup.
- [Rust By Example](#)
Short runnable examples for concepts like structs, enums, traits, modules, error handling, testing, and generics.
- [Rustlings](#)
Small exercises intended to be done alongside the Rust Book.

Study these topics in order:

1. Ownership, borrowing, references.
2. Structs, enums, pattern matching.
3. `Result` , `Option` , and error propagation.
4. Traits and generic bounds.
5. Modules, crates, and visibility.
6. Unit tests.

Then revisit these repo files:

- `lib/src/models/bank.rs`
- `lib/src/shared/error.rs`
- `lib/src/services/bank.rs`

Phase 2: Cargo, Workspaces, and Crate Structure

Use:

- [The Cargo Book](#)
- [Cargo Workspaces](#)

Focus on:

1. Cargo.toml
2. Workspace members.
3. Path dependencies.
4. Feature flags.
5. cargo test , cargo check , package selection.

Apply it to:

- root Cargo.toml
- lib/Cargo.toml
- arma/crate/Cargo.toml

Key concept: this repo separates core logic from extension/runtime code.

forge-lib is reusable domain logic; forge-crate is the adapter around it.

Phase 3: Domain Modeling and Services

Use:

- Rust Book chapters on structs, enums, traits, error handling, and tests.
- Rust By Example sections on custom types, conversion, traits, and error handling.

Study in the repo:

1. lib/src/models/bank.rs
2. lib/src/models/actor.rs
3. lib/src/models/organization.rs
4. lib/src/services/bank.rs
5. lib/src/services/actor.rs
6. lib/src/services/organization.rs

What to learn:

- Why `Money` is a type instead of raw `i64` or `f64` .
- How view models like `PlayerBankProfileView` protect serialized output.
- How services enforce business rules.
- How tests document expected behavior.

Practice:

- Pick one test in `lib/src/services/bank.rs` .
- Read the service method it tests.
- Then read the model methods used by that service.

Phase 4: Repository Pattern and In-Memory Storage

Use Rust Book material on traits and generics, then read:

- `lib/src/repositories/bank.rs`
- `lib/src/repositories/actor.rs`
- `lib/src/repositories/organization.rs`

Supplement with standard library docs:

- [RwLock](#)
- [LazyLock](#)

What to learn:

- Repository traits define storage behavior.
- In-memory repositories support tests and local operation.
- `Arc<RwLock<_>>` allows shared mutable state safely.
- Services depend on traits, not concrete databases.

Phase 5: Serialization and Command Boundaries

Use:

- [Serde overview](#)

Study:

- `lib/src/models/*.rs`

- `arma/crate/src/response.rs`
- `arma/crate/src/transport.rs`
- `arma/crate/src/actor.rs`
- `arma/crate/src/bank.rs`

What to learn:

- `Serialize` and `Deserialize` .
- JSON as the boundary between Arma and Rust.
- Why command handlers parse strings and return strings.
- How large responses are chunked in `transport.rs` .

Practice:

- Trace `bank:deposit` from `arma/crate/src/command.rs` into the `bank` command module, then into `BankService` .

Phase 6: Events and Domain Side Effects

Use Rust Book material on enums, traits, and pattern matching.

Study:

- `lib/src/models/domain_event.rs`
- `lib/src/events/bus.rs`
- `lib/src/events/handlers/*`
- `arma/crate/src/events.rs`

What to learn:

- `DomainEvent` models important business facts.
- `EventBus` dispatches events to handlers.
- `ActorCreated` provisions related resources.
- `ActorDisconnected` triggers cleanup/persistence behavior.

Practice:

- Trace player initialization:
`actor:init -> ActorService::init_or_create -> ActorCreated -> event handlers` .

Phase 7: Async Rust and Persistence

Use:

- [Tokio tutorial](#)
- [SurrealDB Rust crate docs](#)

Study:

- `arma/crate/src/lib.rs`
- `arma/crate/src/persistence/mod.rs`
- `arma/crate/src/persistence/repository.rs`
- `arma/crate/src/persistence/service.rs`
- `arma/crate/src/persistence/surreal.rs`

What to learn:

- Tokio runtime creation.
- Background workers.
- `mpsc` channels.
- Retry loops.
- Cache hydration.
- Write-behind persistence.

Key idea: the game-facing code stays fast by writing to memory first, then queueing database writes.

Phase 8: Arma Extension Layer

Use:

- [arma-rs docs](#)

Study:

- `arma/crate/src/lib.rs`
- `arma/crate/src/command.rs`
- `arma/crate/src/transport.rs`
- one feature group, such as `arma/crate/src/actor.rs`

What to learn:

- `#[arma]`
- `Extension::build()`
- `Group`
- command naming like `actor:init`
- `cdylib`
- why the extension returns strings.

Suggested Books

Use these after the Rust Book, not before:

- *Programming Rust*, O'Reilly: best for deeper systems-level understanding.
- *Rust for Rustaceans*, No Starch: best after you can already read Rust code.
- *Zero to Production in Rust*, Luca Palmieri: useful for service architecture, testing, async, and application structure.

Best Weekly Schedule

Week 1: Rust Book chapters 1-8 plus Rustlings basics.

Week 2: Rust Book traits, generics, error handling, tests.

Week 3: Cargo Book, workspace structure, this repo's crate layout.

Week 4: Models and services in `forge-lib`.

Week 5: Repository traits, in-memory storage, service tests.

Week 6: Serde, JSON boundaries, Arma command adapters.

Week 7: Events and cross-feature workflows.

Week 8: Tokio, persistence queue, SurrealDB, extension runtime.

By the end, you should be able to explain one full use case, such as player initialization or bank transfer, from Arma command input all the way through domain logic, events, repository updates, and persistence.